

Rapport de sécurité — Folio+

Date : 15/07/2026 • **Périmètre** : revue manuelle du repo (migrations SQL/RLS, Edge Functions, GRANT/REVOKE, politiques storage, dépendances npm, code client) + advisor sécurité Supabase.

Méthode : lecture de code + introspection live de la base (`pg_policies` , `pg_proc` , `pg_trigger` , `has_function_privilege`) sur le projet Supabase `rctedezgdxadmjkjeawsj` .

Résumé

Gravité	Nombre	Résumé
Moyenne	2	Injection HTML dans les emails transactionnels ; protection "mot de passe compromis" désactivée
Faible	3	Garde-fou anti-auto-promotion admin reposant sur un seul trigger ; absence de limitation anti-abus sur les endpoints publics ; comparaison de secret non constant-time

Aucune vulnérabilité critique ou élevée trouvée. RLS activée sur toutes les tables, GRANT/REVOKE cohérents, politiques storage correctes, aucun secret commité, `npm audit` propre.

Moyenne

1. Injection HTML dans les emails transactionnels

Description

Dans `supabase/functions/webhook-contact-created/index.ts` et `supabase/functions/webhook-access-request-created/index.ts` , des champs contrôlés par un visiteur anonyme (`name` / `message` du formulaire de contact, `full_name` à l'inscription) sont interpolés **sans échappement** dans le HTML des emails envoyés à l'admin :

```
// webhook-contact-created/index.ts:82-83
<p><strong>${record.name || ""}</strong> (${record.email}) a envoyé un message :</p>
<blockquote>${record.message || ""}</blockquote>
```

```
// webhook-access-request-created/index.ts:88
<p>${visitor?.full_name || visitor?.email || "Un visiteur"} demande l'accès au projet <strong>${
```

Effet

N'importe qui peut soumettre le formulaire public (ou créer un compte) avec `name` / `message` / `full_name` contenant du HTML arbitraire : liens de phishing déguisés, images de tracking (fuite d'IP/read-receipt), mise en page cassée. Pas d'exécution JS classique (la plupart des clients mail retirent `<script>`), mais c'est une injection HTML stockée qui atterrit dans la boîte mail de l'admin.

Correctif recommandé

Échapper `<`, `>`, `&`, `"`, `'` avant interpolation dans les 4 fonctions `webhook-*` (une petite fonction `escapeHtml()` partagée suffit).

Moyenne

2. Protection "mot de passe compromis" désactivée

Description

L'advisor sécurité Supabase (`auth_leaked_password_protection`) signale que la vérification contre HavelBeenPwned.org est désactivée sur l'auth.

Effet

Le flux `AccessRequestModal` crée de vrais comptes avec un mot de passe choisi par un inconnu (self-service, F-12) — sans cette protection, rien n'empêche l'usage d'un mot de passe déjà compromis dans une fuite connue.

Correctif recommandé

Activer l'option dans Dashboard Supabase → Authentication → Politiques (aucun changement de code nécessaire).

Faible

3. Garde-fou anti-auto-promotion admin reposant sur un seul trigger (informationnel)

Description

La policy RLS `user_profiles_update_own` (`WITH CHECK (id = auth.uid())`) ne restreint pas la colonne `role` — à elle seule, elle laisserait n'importe quel utilisateur authentifié s'attribuer `role = 'admin'` via un `UPDATE` direct. Ce qui bloque réellement cette escalade aujourd'hui est le trigger `trg_prevent_role_self_update` (`BEFORE UPDATE OF role ON user_profiles` , fonction `prevent_role_self_update()`), qui lève une exception si un non-admin tente de changer son propre `role` .

Effet

Aucun aujourd'hui — le trigger fonctionne et a été vérifié (`SECURITY DEFINER` , logique correcte : compare `NEW.role / OLD.role` , vérifie `auth.uid() = OLD.id` , vérifie le rôle courant via une sous-requête). Mais c'est un **point de défaillance unique** : si ce trigger est un jour supprimé ou modifié par inadvertance (ex. nettoyage futur de migrations) sans que la dépendance soit connue, l'escalade de privilèges redevient possible via la seule RLS.

Recommandation

Documenter explicitement ce lien dans `CLAUDE.md` (section RLS — pièges à ne pas reproduire), pour qu'une future session ne supprime jamais ce trigger sans réaliser qu'il est la seule protection sur `role`. Alternative plus robuste : ajouter aussi une contrainte au niveau du `WITH CHECK` de la policy elle-même (défense en profondeur), mais non urgent tant que le trigger reste en place.

Faible 4. Absence de limitation anti-abus sur les endpoints publics

Description

`contacts_insert_anyone` (formulaire de contact) et la création de compte via `AccessRequestModal` sont ouverts à tout le monde, sans CAPTCHA ni throttling.

Effet

Combiné au point #1, un script pourrait spammer des demandes/contacts : coût en quota Resend, comptes fantômes en base, boîte mail admin encombrée. Faible priorité pour un portfolio personnel à trafic limité, mais à garder en tête si le formulaire devient une cible (ou si le volume de trafic augmente).

Faible 5. Comparaison du secret webhook non "constant-time" (négligeable)

Description

Dans les 4 fonctions `webhook-*` et `sync-notion-veille`, la vérification du secret partagé utilise une comparaison de chaînes standard (`provided === expected`), potentiellement sensible à une attaque par timing (deviner le secret octet par octet en mesurant les différences de latence).

Effet

Risque pratique quasi nul pour une cible à faible valeur comme celle-ci (le jitter réseau domine largement toute différence de timing mesurable). Mentionné pour complétude.

Recommandation (optionnelle)

Utiliser une comparaison en temps constant si on veut blinder complètement (`crypto.timingSafeEqual` ou équivalent Deno).

Vérifié et sain (pas d'action nécessaire)

- **RLS** activée sur toutes les tables `public.*`.
- **GRANT/REVOKE** cohérents sur les fonctions sensibles : `dispatch_webhook`, `get_webhook_dispatch_secret`, `handle_new_user`, `prevent_role_self_update`,

`create_access_request_notifications` — non exécutables par `anon / authenticated` (vérifié via `has_function_privilege`).

`get_my_role / get_public_cal_username / project_deletion_status` sont intentionnellement publiques et ne fuient aucune donnée au-delà de ce qui est déjà public (vérifié le contenu de chaque fonction).

- **`soft_delete_project`** : callable par `anon / authenticated` au niveau GRANT, mais protégée en pratique car `SECURITY INVOKER` (pas `DEFINER`) — les `UPDATE / DELETE` internes sont filtrés par RLS selon l'appelant, et la fonction vérifie explicitement qu'une ligne a bien été affectée avant de retourner un succès.
- **Policies storage** (`project-thumbnails` , `designer-photos`) : écriture admin-only, pas de policy `SELECT` superflue (pas de listing public possible).
- **`anonymize-rgpd`** : la cible est toujours résolue depuis le JWT de l'appelant (`auth.getUser()`), jamais depuis le corps de la requête — un utilisateur ne peut anonymiser que son propre compte.
- **Secrets** : aucun `.env / clé/token` commité (`.gitignore` correct, vérifié via `git ls-files`).
- **Dépendances** : `npm audit --production` → 0 vulnérabilité connue.
- **XSS côté client** : aucun `dangerouslySetInnerHTML` actif (seule occurrence dans le code mort `src/components/ui/chart.tsx` , jamais importé) ; `MarkdownContent.tsx` (react-markdown) n'utilise aucun plugin de rendu HTML brut (pas de `rehype-raw`).